SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CO2-DEBUGGING QUESTIONS

NAME: P. DEEKSHANA

REG NO: 192425104

COURSE CODE: CSA0601

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

(a) Reproduce and Identify the Defects

1. Introduction

Debugging is the process of identifying and correcting errors in a program. Optimisation improves program performance without changing its correct output.

The SentinelOps module performs four operations: sorting security alerts, checking blocked IP addresses, finding attack signatures in logs, and identifying the closest pair of login locations. The original program contains nine defects that cause incorrect outputs and unnecessary computations.

2. Identification of Defects

Defect 1: Incorrect insertion-sort condition

```
while j > 0 and alerts[j][1] < key[1]:
```

The condition prevents comparison with index 0. Correct it to j >= 0 so that higher-severity alerts can reach the beginning.

Defect 2: Incorrect insertion position

```
alerts[j] = key
```

The key must be inserted at j + 1. Otherwise, an existing alert may be overwritten.

Defect 3: Incorrect blocklist return value

```
if blocklist[i] == ip:
    return False
```

The function must return True when the IP address is found.

Defect 4: Incorrect else placement

The else statement returns False after the first non-matching address. The final return False must be outside the loop.

Defect 5: Incorrect string-matching boundary

```
for i in range(n - m):
```

This excludes the final valid starting position. Correct it to range(n - m + 1).

Defect 6: Incorrect match condition

```
if j == m - 1:
```

This may report an incomplete match. Check j == m after all pattern characters have matched.

Defect 7: Incorrect minimum-distance initialisation

```
min_dist = 0
```

Initialise it to float("inf") so that the first valid distance can become the minimum.

Defect 8: Incorrect distance comparison

```
if d > min_dist:
```

This selects the largest distance. Replace > with < to find the closest pair.

Defect 9: Repeated and self-pair comparisons

```
for j in range(len(events)):
```

This compares events with themselves and repeats pairs. Use range(i + 1, len(events)) to examine each distinct pair once.

3. Expected Outputs

Function	Correct output
Sorting alerts	A2:9, A4:9, A3:5, A1:3, A5:1

Blocklist search	True, True, False
Signature matching	[0, 8]
Closest login pair	(0, 5), distance approximately 1.4142 km

(b) Explain the Root Causes and Debug Systematically

1. Corrected Alert-Sorting Function

Insertion sort arranges alerts in descending severity order while preserving the arrival order of equal-severity alerts.

```
def sort_alerts(alerts):
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1

        while j >= 0 and alerts[j][1] < key[1]:
            alerts[j + 1] = alerts[j]
            j -= 1

        alerts[j + 1] = key

    return alerts
```

Trace:

- Initial severity order: 3, 9, 5, 9, 1
- After inserting A2: 9, 3, 5, 9, 1
- After inserting A3: 9, 5, 3, 9, 1
- After inserting A4: 9, 9, 5, 3, 1
- After inserting A5: 9, 9, 5, 3, 1

Output:

```
[('A2', 9), ('A4', 9), ('A3', 5),
 ('A1', 3), ('A5', 1)]
```

All five alerts are preserved, and equal-severity alerts retain their arrival order.

2. Corrected Blocklist Function

```
def is_blocked(blocklist, ip):
    for address in blocklist:
        if address == ip:
            return True
    return False
```

The function checks every address until it finds a match. If no match exists, it returns False after the loop.

Output:

```
10.0.0.9: True
10.0.0.5: True
8.8.8.8: False
```

3. Corrected Signature-Matching Function

```
def find_signature(log, pattern):
    n = len(log)
    m = len(pattern)
    hits = []

    if m == 0 or m > n:
        return hits

    for i in range(n - m + 1):
        j = 0

        while j < m and log[i + j] == pattern[j]:
            j += 1

        if j == m:
            hits.append(i)

    return hits
```

For the log FAILFAIXFAIL and pattern FAIL, complete matches occur at indices 0 and 8. The candidate at index 4 is rejected because its final character does not match.

Output:

```
[0, 8]
```

4. Corrected Closest-Login Function

```python
import math

def closest_logins(events):
    if len(events) < 2:
        return None, float("inf")

    min_dist = float("inf")
    pair = None

    for i in range(len(events)):
        for j in range(i + 1, len(events)):
            dx = events[i][0] - events[j][0]
            dy = events[i][1] - events[j][1]
            d = math.sqrt(dx * dx + dy * dy)

            if d < min_dist:
                min_dist = d
                pair = (i, j)

    return pair, min_dist
```

For locations (2, 3) and (3, 4):

$$d = \sqrt{(3-2)^2 + (4-3)^2} = \sqrt{2}$$

The minimum distance is approximately 1.4142 km.

5. Boundary Testing

Test	Input	Expected result
Empty alert list	[]	[]
Equal severities	[(“A”,5),(“B”,5)]	Original order preserved
Empty blocklist	[]	False
Pattern at log end	“ABCFAIL”, “FAIL”	[3]
Pattern longer than log	“ABC”, “ABCDEF”	[]
Empty pattern	“”	[]
No login events	[]	No pair
One login event	[(2,3)]	No pair

These tests confirm that the corrected functions handle both normal and boundary inputs.

(c) Optimise the Corrected Module

1. Optimising Alert Sorting

Insertion sort has a worst-case time complexity of $O(n^2)$. For 50,000 alerts, it can require up to 1,249,975,000 comparisons.

Python's built-in stable sorting is more suitable for large batches.

```
def sort_alerts(alerts):
    return sorted(
        alerts,
        key=lambda alert: alert[1],
        reverse=True
    )
```

Timsort has worst-case time complexity $O(n \log n)$and preserves the order of equal-severity alerts.

2. Optimising Blocklist Search

Sequential search requires $O(b)$time per query. A set improves repeated lookups.

```
def create_blocklist(blocklist):
    return set(blocklist)

def is_blocked(blocklist_set, ip):
    return ip in blocklist_set
```

Set construction takes expected $O(b)$time, while membership checks take average $O(1)$time.

3. Optimising Signature Matching

The corrected brute-force algorithm stops at the first mismatch, avoiding unnecessary character comparisons.

Its worst-case complexity remains $O(Lm)$, where $L$is the log length and $m$is the pattern length. For very long logs, the Knuth–Morris–Pratt algorithm can improve this to $O(L + m)$.

4. Optimising Closest-Pair Search

```
def closest_logins(events):
    n = len(events)

    if n < 2:
        return None, float("inf")

    min_dist_sq = float("inf")
    pair = None
```

```
    for i in range(n):
        for j in range(i + 1, n):
            dx = events[i][0] - events[j][0]

            if dx * dx >= min_dist_sq:
                continue

            dy = events[i][1] - events[j][1]
            dist_sq = dx * dx + dy * dy

            if dist_sq < min_dist_sq:
                min_dist_sq = dist_sq
                pair = (i, j)

    return pair, math.sqrt(min_dist_sq)
```

This version examines each pair once, avoids repeated square-root calculations, and skips pairs that cannot improve the current minimum. Its worst-case time complexity remains $O(n^2)$.

(d) Analyse Before and After Optimisation

1. Complexity Comparison

Function	Before optimisation	After optimisation
Alert sorting	Insertion sort: $O(n^2)$worst case	Timsort: $O(n \log n)$worst case
Blocklist search	$O(b)$per query	Average $O(1)$per query
Signature matching	$O(Lm)$	$O(Lm)$; KMP alternative: $O(L + m)$
Closest-pair search	$O(n^2)$	$O(n^2)$worst case, with fewer calculations

2. Closest-Pair Operation Counts

The original code examines n^2ordered pairs. The corrected algorithm examines $n(n-1)/2$distinct pairs.

Login events	Original iterations	Corrected iterations
1,000	1,000,000	499,500
4,000	16,000,000	7,998,000
8,000	64,000,000	31,996,000

The corrected algorithm reduces pair-loop iterations by approximately half.

3. Benchmarking Code

```python
import random
import time

def count_pairs(events):
    n = len(events)
    count = 0

    for i in range(n):
        for j in range(i + 1, n):
            dx = events[i][0] - events[j][0]

            if dx * dx >= float("inf"):
                continue

            dy = events[i][1] - events[j][1]
            distance_sq = dx * dx + dy * dy
            count += 1

    return count

for n in [1000, 4000, 8000]:
    random.seed(42)
    events = [
        (random.randint(0, 100000),
         random.randint(0, 100000))
        for _ in range(n)
    ]

    start = time.perf_counter()
    count = count_pairs(events)
    elapsed = time.perf_counter() - start

    print(n, count, round(elapsed, 4), "seconds")
```

This program measures the execution time and number of distinct-pair distance calculations. Actual timings depend on the computer and input data.

4. Insertion-Sort Comparison

Insertion sort performs well on nearly sorted input because fewer elements need to be shifted. Randomly ordered input generally requires more comparisons and shifts. For large batches, built-in sorting is usually preferable.

Conclusion: Theoretical analysis explains the growth in operations, while benchmarking demonstrates actual performance.

(e) Deliver Production-Quality Code

Final Python Module

```python
import math


def sort_alerts(alerts):
    """Sort alerts by severity, highest first, preserving equal-severity order."""
    return sorted(alerts, key=lambda alert: -alert[1])


def create_blocklist(blocklist):
    """Create a set for efficient repeated IP-address lookups."""
    return set(blocklist)


def is_blocked(blocklist_set, ip):
    """Return True if the IP address exists in the blocklist."""
    return ip in blocklist_set


def find_signature(log, pattern):
    """Return every starting position where the pattern occurs in the log."""
    n, m = len(log), len(pattern)

    if m == 0:
        return list(range(n + 1))

    if m > n:
        return []

    hits = []

    for i in range(n - m + 1):
        if log[i:i + m] == pattern:
            hits.append(i)

    return hits


def closest_logins(events):
    """Return the closest pair of login locations and their distance."""
    if len(events) < 2:
        return None, math.inf

    min_squared = math.inf
    pair = None

    for i in range(len(events)):
        for j in range(i + 1, len(events)):
            dx = events[i][0] - events[j][0]

            if dx * dx >= min_squared:
                continue

            dy = events[i][1] - events[j][1]
            distance_squared = dx * dx + dy * dy

            if distance_squared < min_squared:
                min_squared = distance_squared
                pair = (i, j)

    return pair, math.sqrt(min_squared)


def run_tests():
    """Test normal inputs and boundary cases for all functions."""

    alerts = [
        ("A1", 3), ("A2", 9), ("A3", 5),
        ("A4", 9), ("A5", 1)
    ]

    assert sort_alerts(alerts) == [
        ("A2", 9), ("A4", 9), ("A3", 5),
        ("A1", 3), ("A5", 1)
    ]

    blocklist = create_blocklist([
        "10.0.0.5", "10.0.0.9", "172.16.4.2"
    ])
```

```
    assert is_blocked(blocklist, "10.0.0.9") is True
    assert is_blocked(blocklist, "8.8.8.8") is False
    assert is_blocked(create_blocklist([]), "8.8.8.8") is False

    assert find_signature("FAILFAIXFAIL", "FAIL") == [0, 8]
    assert find_signature("ABCFAIL", "FAIL") == [3]
    assert find_signature("ABC", "LONGER") == []
    assert find_signature("ABC", "D") == []
    assert find_signature("ABC", "") == [0, 1, 2, 3]

    events = [(2, 3), (12, 30), (40, 50),
              (5, 1), (12, 10), (3, 4)]

    pair, distance = closest_logins(events)
    assert pair == (0, 5)
    assert abs(distance - math.sqrt(2)) < 1e-9

    assert closest_logins([]) == (None, math.inf)
    assert closest_logins([(1, 2)]) == (None, math.inf)

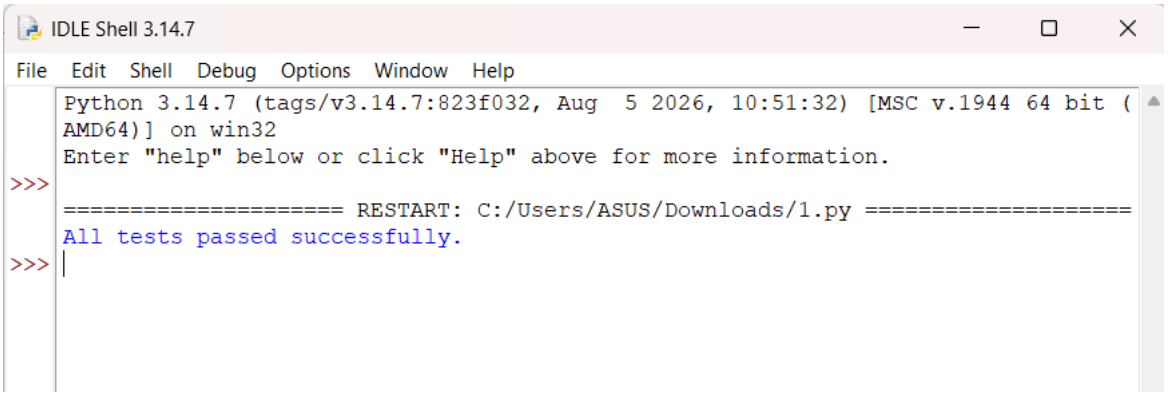
    print("All tests passed successfully.")


if __name__ == "__main__":
    run_tests()
```

Expected output:

All tests passed successfully.

Output:



Conclusion

The SentinelOps module was corrected by identifying nine defects and fixing the sorting, searching, string-matching, and closest-pair functions. Boundary tests verify that the corrected algorithms handle empty inputs, equal severities, patterns at the end of logs, and insufficient login events.

Optimisations improve performance through stable sorting, set-based IP lookups, early termination during string matching, and squared-distance comparisons. Complexity analysis and benchmarking demonstrate how these changes reduce unnecessary operations while preserving correct results.